

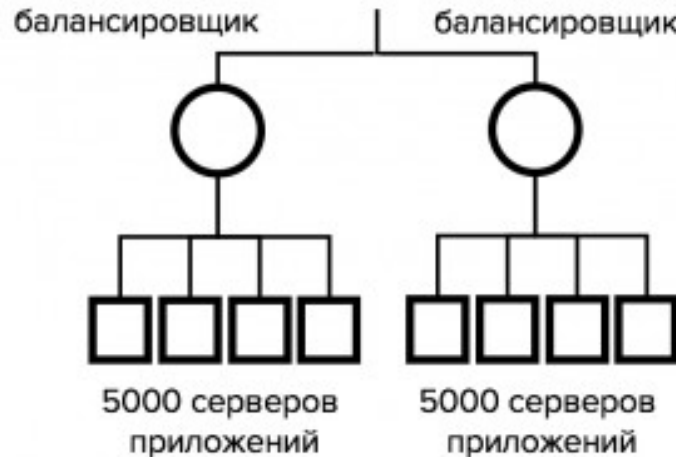


Шаблоны регистрации, обнаружения и использования микросервисов



Проблема

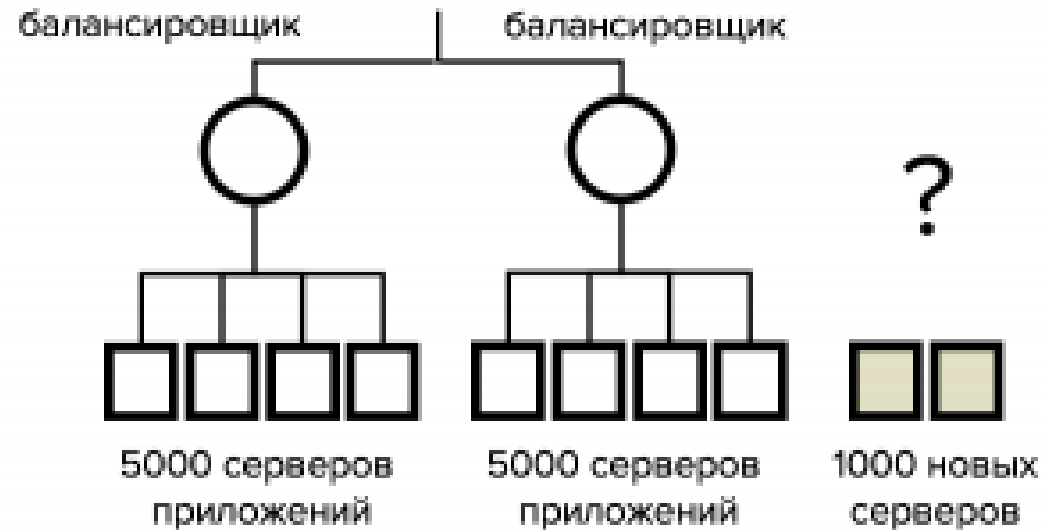
Допустим, приложение, обеспечивающее работу вашего магазина, работает на кластере из тысячи серверов (остальные 9.000 рулят базами данных, бэкапами и кэшированием). Серверы спрятаны за двумя балансировщиками нагрузки (два балансера на одном доменном имени — тоже своего рода балансировка, ведь при обращении по доменному адресу юзер может попасть как на первый балансир, так и на второй).





Проблема

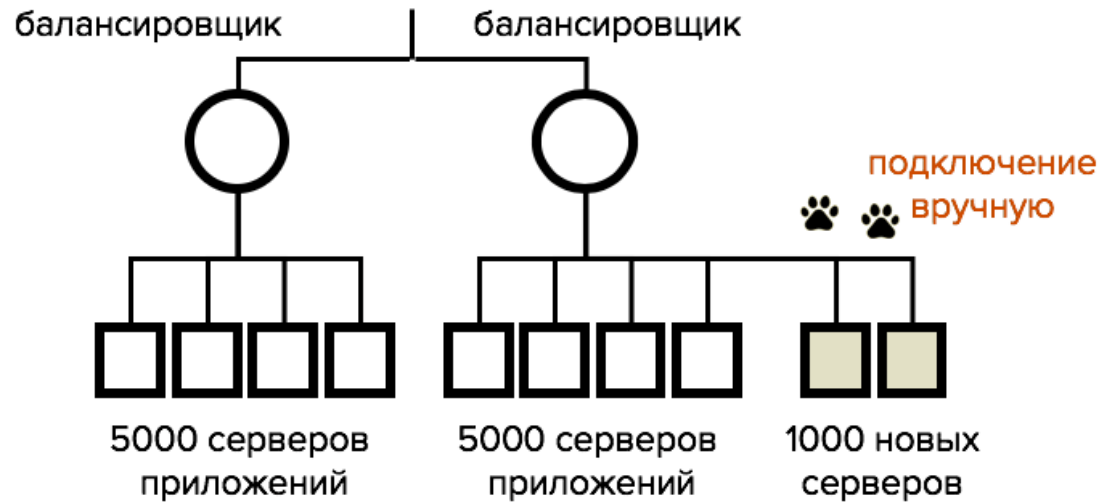
Вы решаете нарастить мощность вашего магазина, добавив еще серверов с приложениями.





Проблема

Осталось сообщить балансировщику нагрузки о том, что у него в кластере приложения в подчинении появилось еще, скажем, 1000 серверов. Для этого нужно пойти в настройки балансировщика и вписать там руками адреса новых машин.





Проблема

Конечно, можно разок напрячься и руками вписать 1000 адресов в конфиги балансировщика. А что делать, если балансировщиков 500, а серверов приложений у них в подчинении 100 000? И они еще и меняют адреса, умирают, переезжают?



Проблема

- Сервисы и микросервисы стали ответом на растущую сложность серверного ПО, предлагая следующие преимущества:
 - Масштабирование
 - Переиспользование
 - Свобода технологий

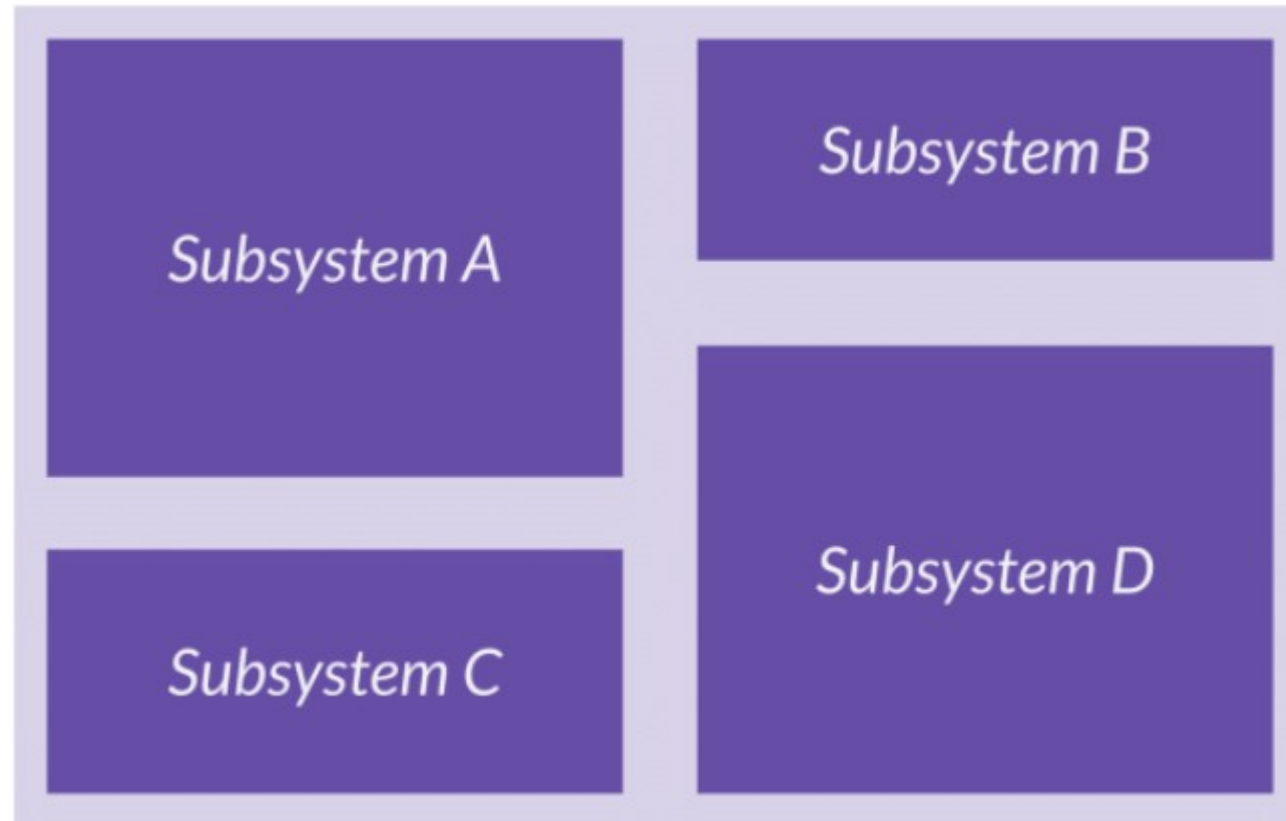


НО!

- Проблема:
 - Сложность работы в области межсервисного взаимодействия, управления конфигурацией и сегментации сети.

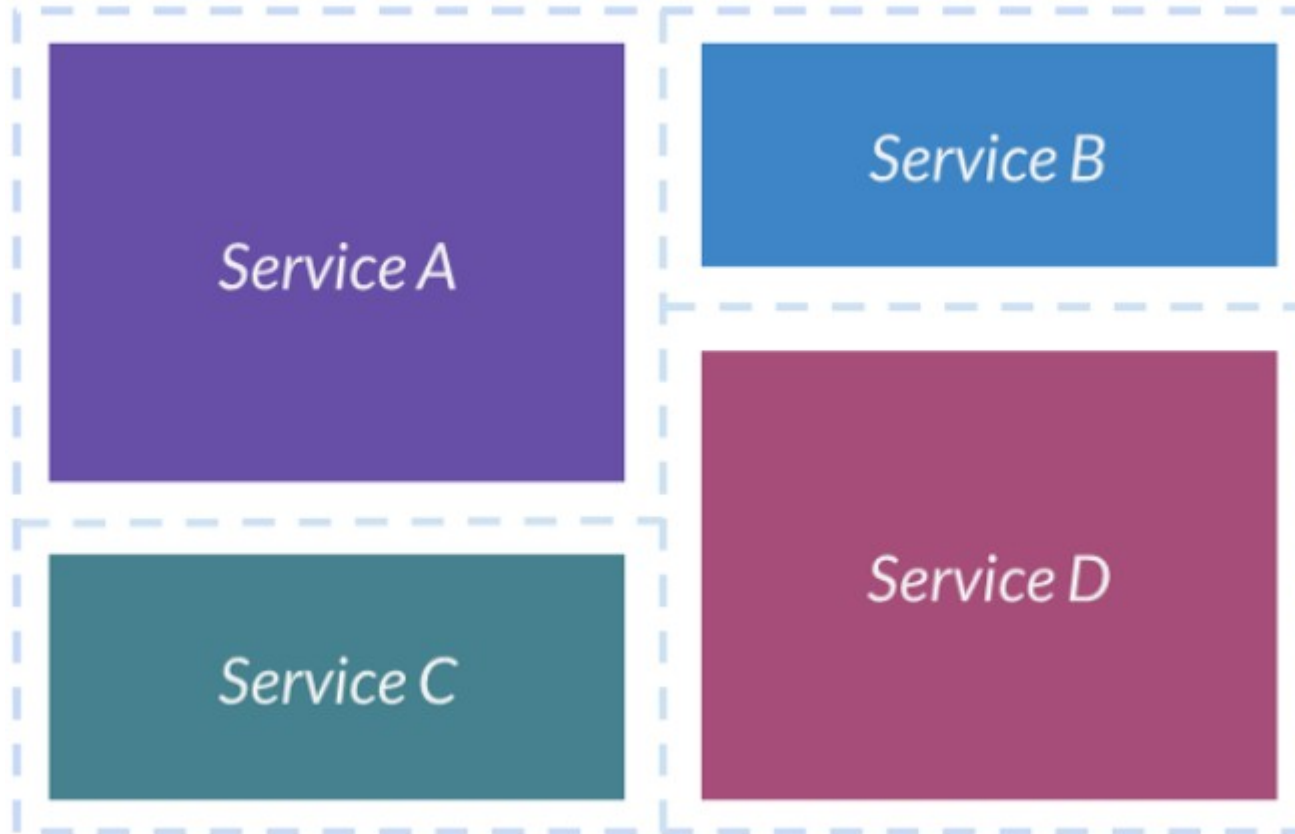


Было





Стало





Решение!

Проблемы управления инфраструктурой в таких масштабах с успехом решает специальный журнал, в который серверы и компоненты инфраструктуры сообщают о своих адресах и состоянии, как только они появляются в кластере. Любые системы, которым нужно знать обо всех серверах, в курсе, что всё это можно узнать в этом журнале (если, конечно, у них есть на это права). Система, которая ведёт такой журнал, называется Service Discovery (SD) или Service Registry.



Решение!

- Сервисы должны быть зарегистрированы в общедоступном реестре служб, чтобы их можно было находить во время выполнения и соответствующим образом использовать
- В динамичной и распределенной среде сервисы изменяются, поэтому задача обнаружения сервисов во время выполнения должна быть облегчена с помощью таких доступных по сети реестров и репозиторий



Микросервисы и их взаимодействие

- Популярные решения, такие как Kubernetes и Marathon (<https://mesosphere.github.io/marathon/docs/service-discovery-load-balancing.html>), запускают прокси-серверы на каждом хосте
- Прокси-серверы маршрутизируют запросы к службам в кластере

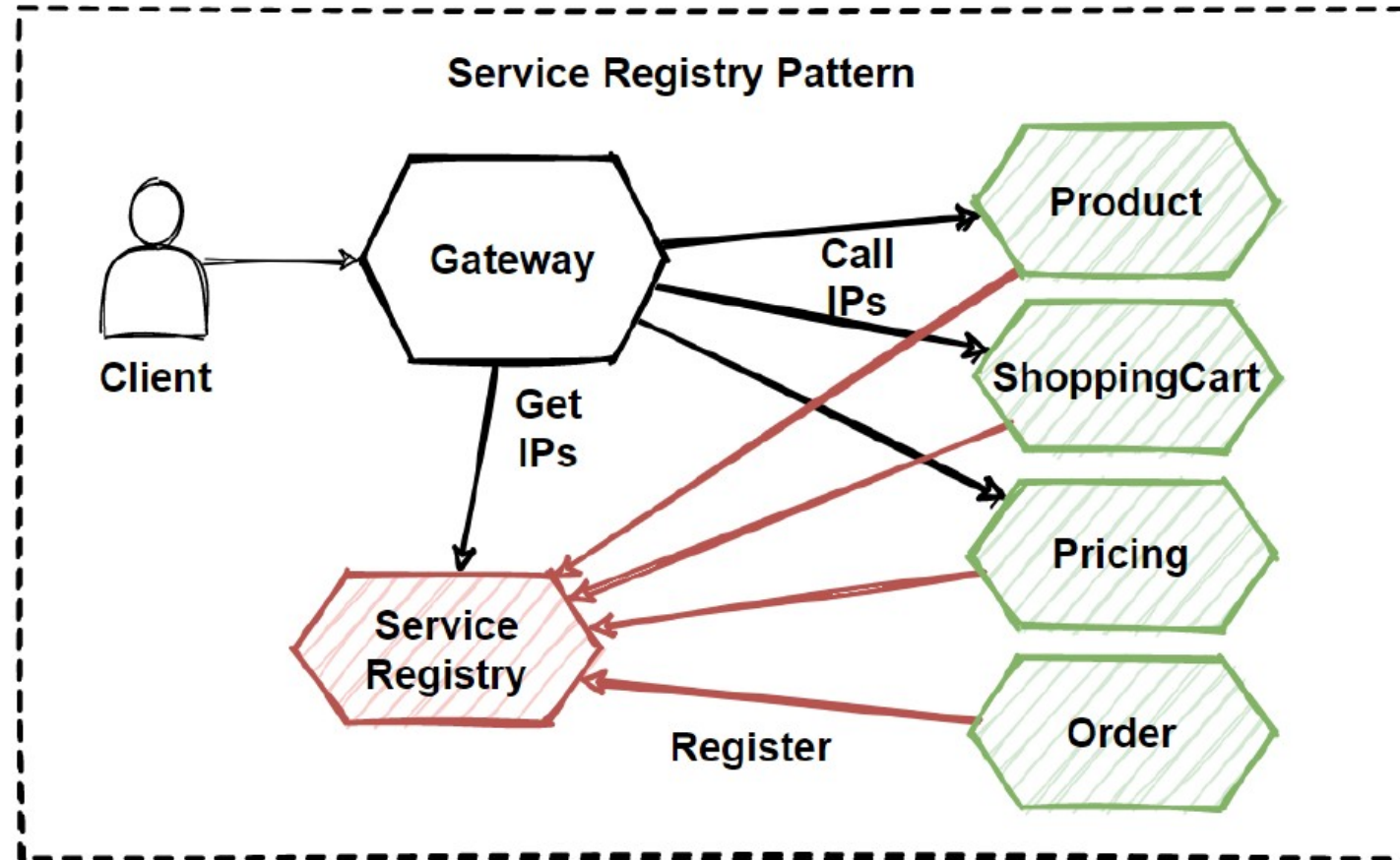


Шаблон «Service Registry»

- Реестр сервисов содержит все необходимые ссылки на каждый сервис
- Каждый разработанный сервис должен быть зарегистрирован в реестре сервисов, чтобы быть найденным, связанным и полноценно использованным
- Для соединения с другими сервисами сервис должен сначала подключиться к реестру сервисов, чтобы получить все сведения о доступности и использовании других сервисов



Шаблон «Service registry»



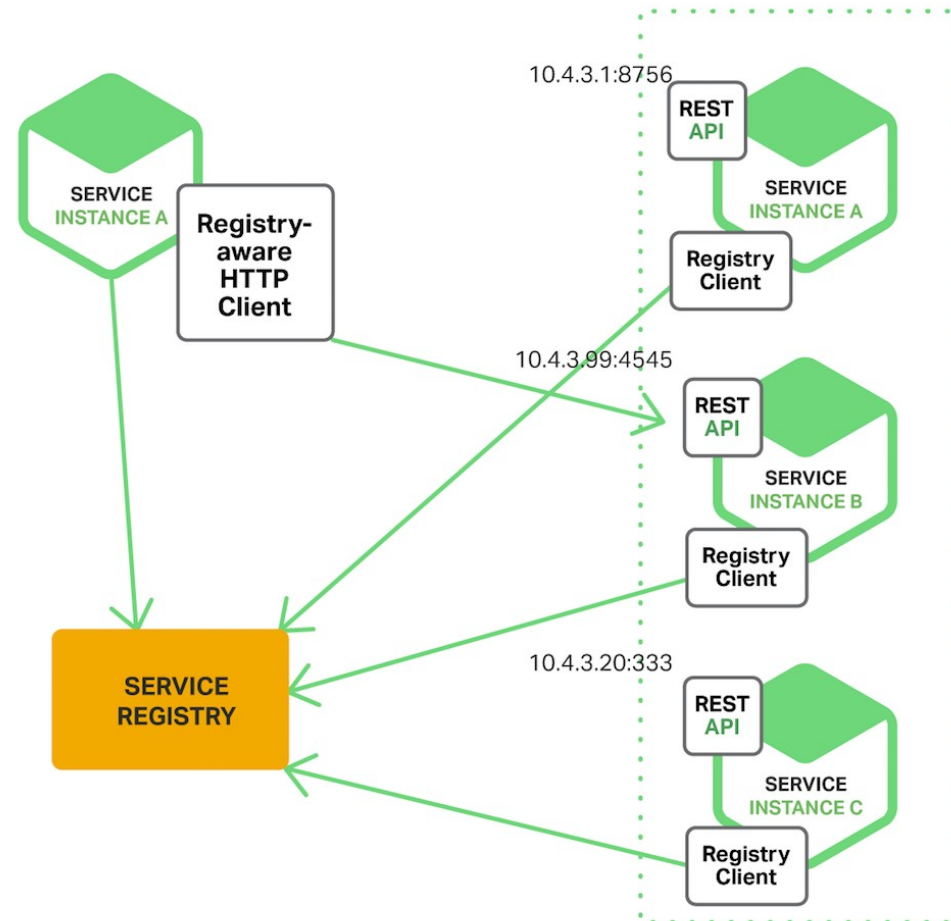


Шаблон «Service registry»

- Перед регистрацией сервиса в реестре, вы должны определить метаданные, которые будут использоваться для его идентификации и поиска. Это могут быть такие данные, как имя сервиса, версия, IP-адрес, порт и другие параметры, которые будут использоваться для обнаружения и взаимодействия с сервисом.
- Когда метаданные определены, сам сервис должен зарегистрироваться в сервис-реестре. Это делается путем отправки информации о себе (метаданные) в реестр. Существует несколько способов сделать это, и выбор метода зависит от используемых технологий.
 - Например, для приложений, работающих в контейнерах, часто используются библиотеки и агенты, которые автоматически регистрируют сервис в реестре при запуске.

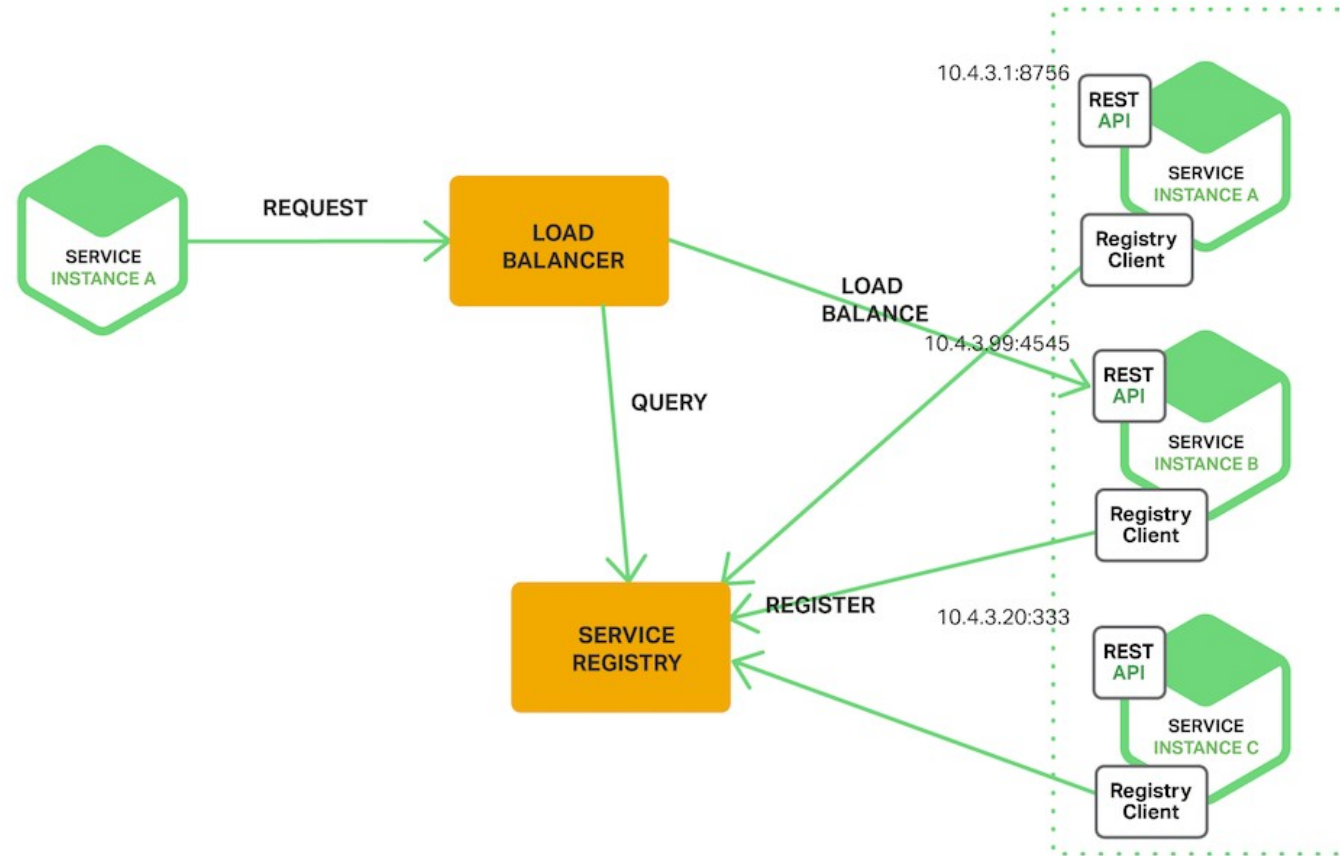


Шаблон «Service registry»





Шаблон «Service registry»





Шаблон «Service registry»

- Регистрация в сервис-реестре часто предполагает также периодическое обновление информации о сервисе, чтобы отслеживать его доступность и состояние. Это важно для того, чтобы другие микросервисы могли надежно обращаться к нему.
- Другие микросервисы могут обращаться к сервису, зарегистрированному в реестре, путем обнаружения его метаданных из реестра. Это позволяет установить соединение с сервисом, использовать его и обеспечить надежную коммуникацию.



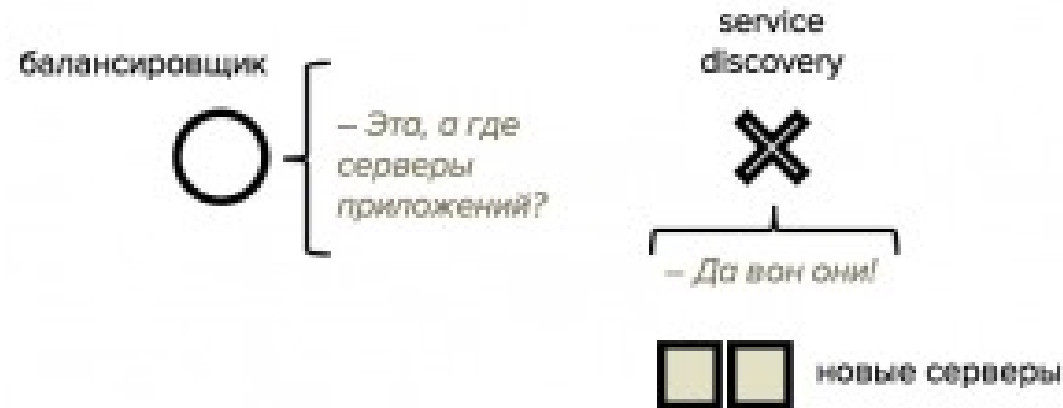
В итоге

Итак, вы засунули в ваш кластер приложения 1000 новых серверов приложений. Всем им админы задают адрес системы Service Discovery (например, с помощью системы автоматизации развертывания серверов), у которой надо прописаться, и немедленно шлют туда сообщение типа “Привет, я сервер приложения, я готов к работе, и вот мой адрес”. Service Discovery принимает подобные запросы не только от серверов приложений, но и от любых инфраструктурных компонентов в сети. В итоге получается, что этот сервис-всезнайка может в любой момент точно сказать, какие серваки есть в сети, что они делают и где живут.



В итоге

Балансировщик нагрузки, вместо обращения к своим собственным файлам конфигов, раз в N секунд обращается в Service Discovery и запрашивает там список живых серверов приложений. После очередного цикла работы балансировщик получит обновленный список серверов приложений и начнет распределять нагрузку по всем новым машинам.





В итоге

- В Service Discovery можно запихивать любые компоненты инфраструктуры — базы данных, серверы кэширования, микросервисы, резервные машины.
- При любых изменениях в системе все заинтересованные части получают обновленные состояния инфраструктуры и изменяют свою работу под новые условия. А заинтересованных агентов обычно много — балансировщики желают знать свои кластеры приложений, приложения хотят знать свои СУБД и системы кэширования, системы мониторинга хотят знать сервисы, подлежащие наблюдению.
- Изменения состояний — это не только появление новых серверов, но и их выход из строя, переезд, перенос между подсетями.

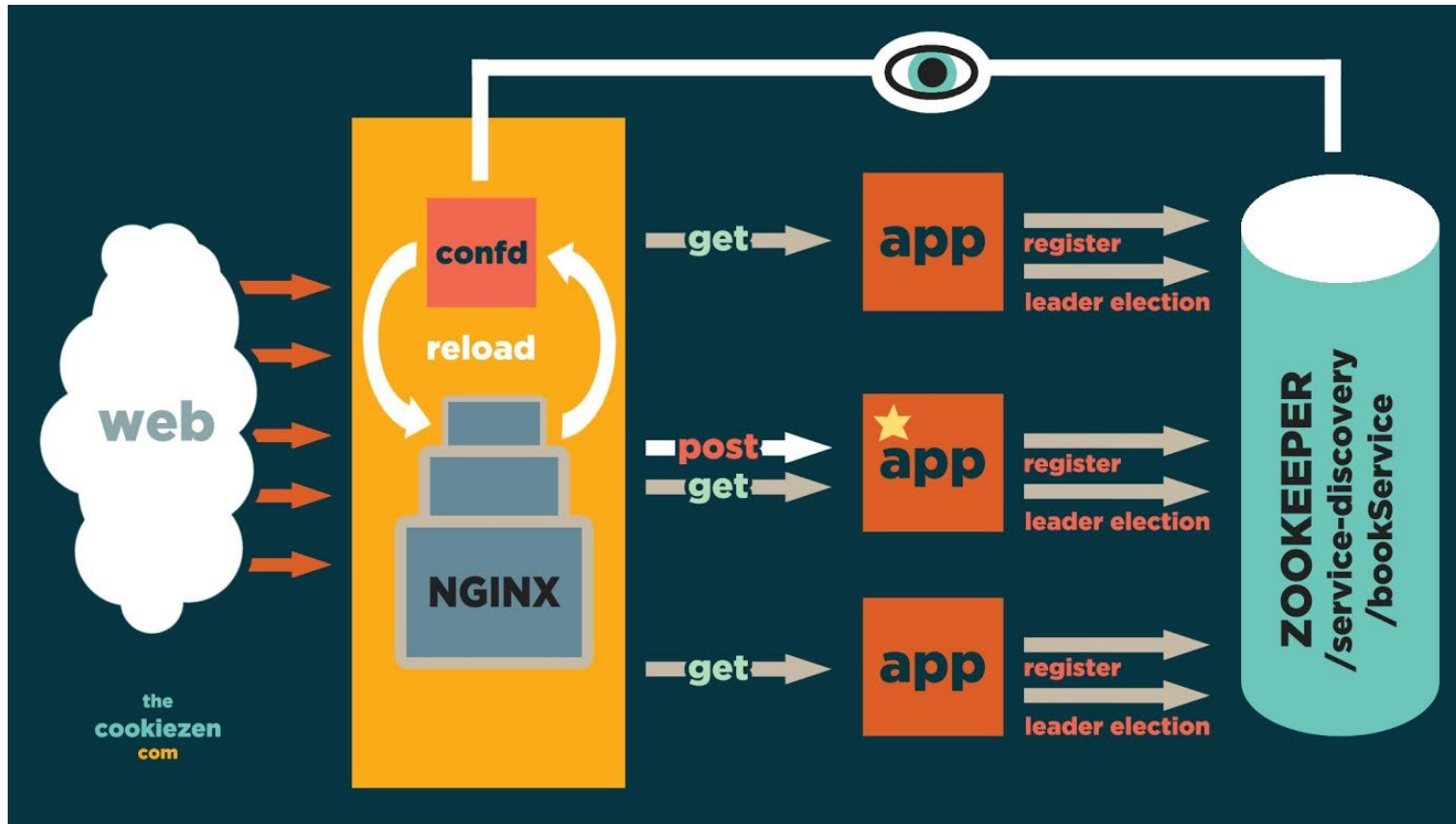


Какие бывают Service Discovery?

- ZooKeeper
- Etcd
- Consul
- Kubernetes

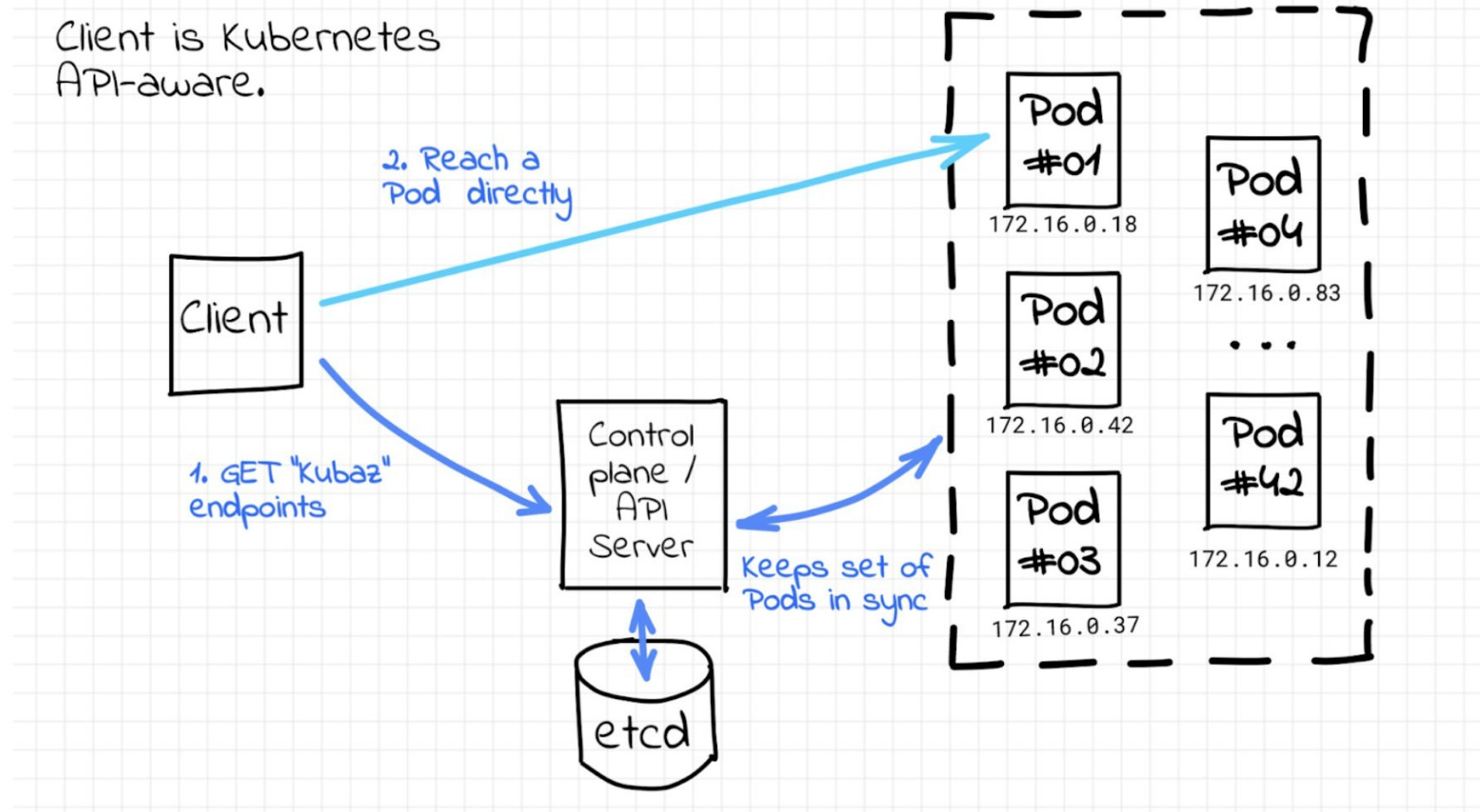


ZooKeeper



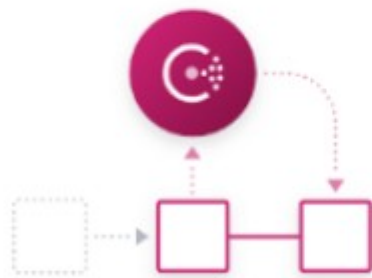


Etcd





Consul



Service Discovery



Service Configuration



Service Segmentation



Consul

- Исторический контекст создания Consul крайне важен для понимания его роли в современной инфраструктуре. Consul был представлен миру в 2014 году компанией HashiCorp как решение для обнаружения сервисов и конфигурации.
- В то время облачные вычисления и сервис-ориентированная архитектура становились мейнстримом, и компании сталкивались с серьезной проблемой - как правильно организовать маршрутизацию между сервисами и обрабатывать сбои в распределенной системе.



Consul

- Consul произвел революцию в индустрии, объединив DNS-based service discovery с надежной системой обнаружения сбоев. Принцип работы был достаточно прост - сервис регистрировался в Consul, и другие сервисы могли использовать его Consul DNS запись для маршрутизации.
- Например, если у вас был фронтенд сервис, он становился доступен по адресу `frontend.service.consul`. Более того, Consul отслеживал работоспособность сервисов используя алгоритм gossip под названием Serf и систему проверок здоровья. Если узел или сервис выходил из строя, Consul быстро обнаруживал это и удалял его из DNS.



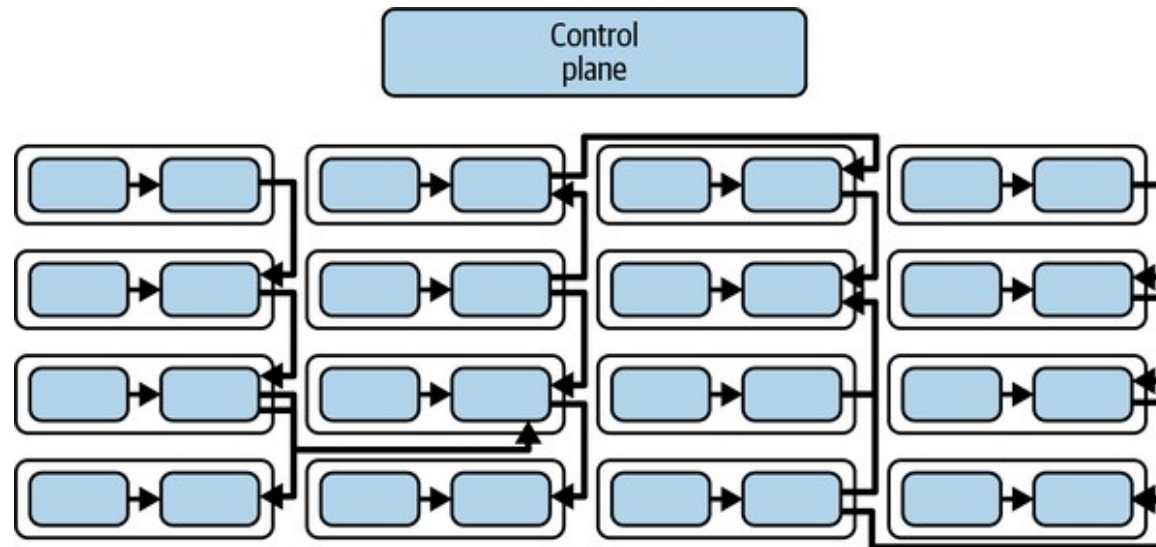
Consul

- С течением времени сервис-ориентированная архитектура эволюционировала в микросервисную архитектуру, где сервисы становились все меньше и меньше. Это привело к появлению Docker и оркестраторов контейнеров, таких как Kubernetes. Эти изменения в индустрии повлияли на развитие Consul двумя важными способами.
- Во-первых, простого DNS service discovery стало недостаточно. Разработчикам требовалось больше сетевых функций в области безопасности, наблюдаемости, надежности и управления трафиком для эффективного управления всеми этими сервисами. Также становилось все сложнее реализовывать эти функции в коде сервисов, поскольку количество сервисов постоянно росло.



Consul

- Во-вторых, Kubernetes значительно упростил запуск sidecar прокси благодаря модели pod, где несколько контейнеров могут работать вместе в частной сети. Эти изменения создали основу для современной технологии service mesh, какой мы ее знаем сегодня.





Consul

- Команда Consul в HashiCorp внимательно следила за этими тенденциями, и в 2018 году они выпустили функцию service mesh под названием Consul Connect.
- Изначально Consul Connect фокусировался на безопасной коммуникации между сервисами и добавил поддержку sidecar прокси для шифрования трафика. С тех пор Consul развился в полноценный service mesh с функциями не только в области безопасной коммуникации, но также наблюдаемости, надежности и управления трафиком.

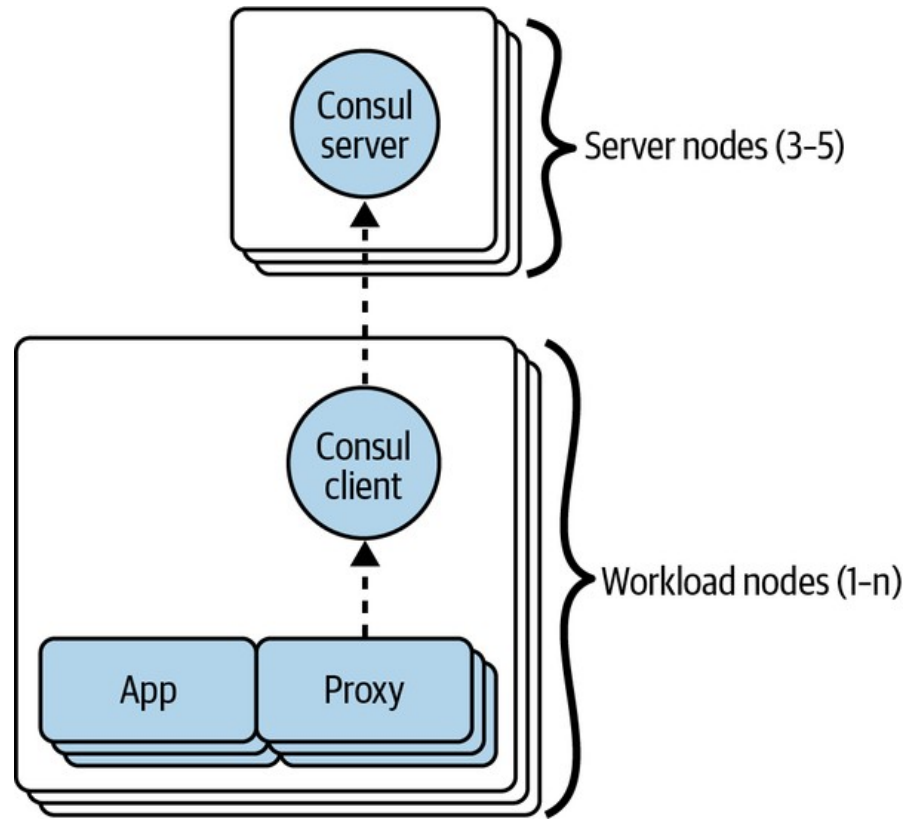


Архитектура Consul

- Рассмотрим архитектуру Consul.
- Установка Consul состоит из трех основных компонентов:
 - Consul серверов, Consul клиентов и sidecar прокси.
 - Sidecar прокси общаются со своими локальными Consul клиентами, а Consul клиенты общаются с Consul серверами.
 - Обычно в производственной среде используется три или пять Consul серверных узлов, а количество рабочих узлов может достигать нескольких тысяч.



Архитектура Consul





Consul серверы

- Consul серверы представляют собой базу данных всей системы.
- В них хранится каталог, который содержит список всех узлов и сервисов, конфигурации, статусы здоровья и другая важная информация.
- Для производственного использования Consul необходимо запускать несколько серверов, хотя для тестирования достаточно одного. Использование нескольких серверов обеспечивает высокую доступность и сохранность данных - если один сервер выходит из строя, остальные серверы продолжают обрабатывать запросы, и никакие данные не теряются.



Consul серверы

- Важно отметить, что всегда следует запускать нечетное количество серверов, например, три или пять. Запуск четного количества серверов не увеличивает отказоустойчивость Consul, и дополнительный сервер просто расходует ресурсы впустую.
- Для нормального функционирования Consul требуется большинство работающих серверов. При работе трех серверов большинством являются два сервера, поэтому система может выдержать отказ одного сервера. При работе четырех серверов большинством являются три сервера, то есть система по-прежнему может выдержать отказ только одного сервера.



Consul серверы

- Consul использует протокол под названием Raft для обеспечения согласованности данных между серверами.
- Это особенно важно в случаях, когда возникают проблемы с сетью и серверы могут потерять связь друг с другом.
- Raft выполняет две важные функции: выбор лидера и репликацию данных. При выборе лидера один из Consul серверов назначается главным, и все записи в систему проходят через него. Это предотвращает конфликты, которые могли бы возникнуть, если бы разные значения записывались одновременно на разные серверы.
- Репликация обеспечивает копирование всех записей с сервера-лидера на остальные серверы, гарантируя сохранность данных даже при смене лидера.



Consul серверы

- На физическом уровне Consul серверы хранят свои данные на диске. В среде Kubernetes они разворачиваются как StatefulSet и используют PersistentVolumes для хранения данных.
- При работе на виртуальных машинах Consul серверы должны быть развернуты с дисковыми томами, которые не теряются при перезагрузке узла.
- В производственной среде рекомендуется выделять отдельный узел для каждого Consul сервера, чтобы избежать проблем с конкуренцией за ресурсы.



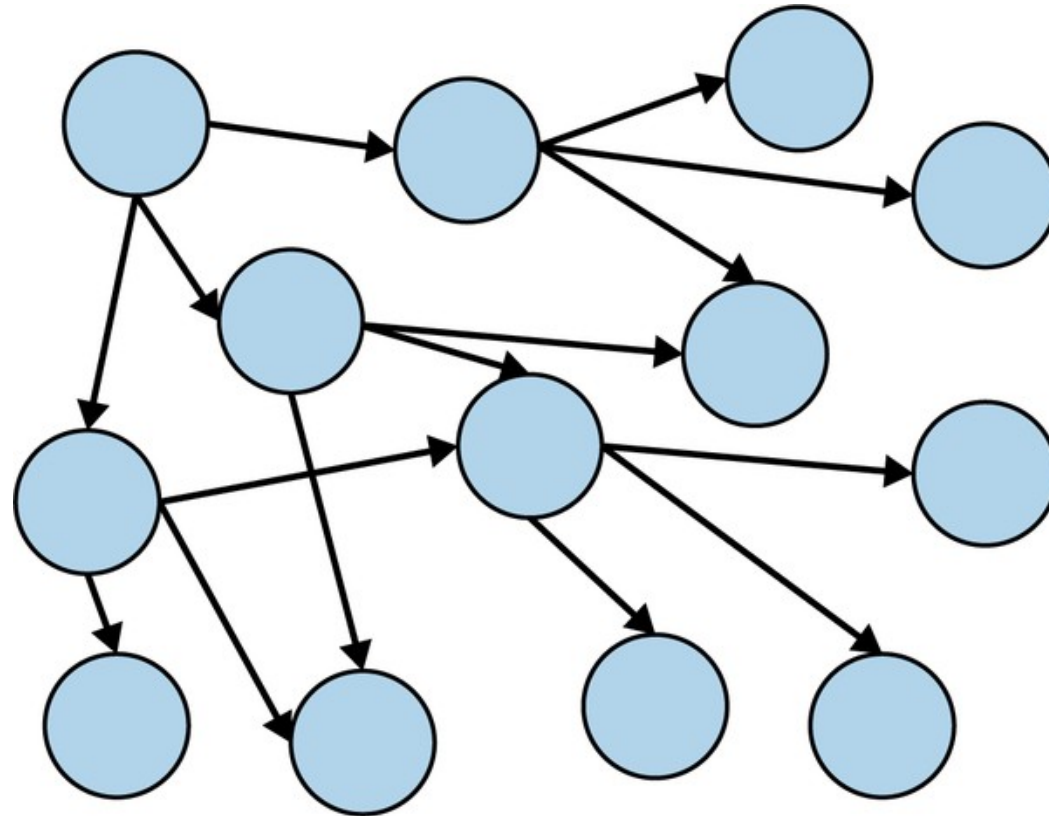
Consul клиенты

- Consul клиенты запускаются на каждом рабочем узле в кластере. Их основная задача - отслеживать здоровье сервисов, работающих на их узле, а также здоровье других узлов в кластере. Consul клиенты постоянно отправляют эту информацию Consul серверам, чтобы поддерживать каталог в актуальном состоянии.
- Теоретически, Consul серверы могли бы сами проверять здоровье всех сервисов и узлов в кластере, и необходимость в Consul клиентах отпала бы.
 - Однако на практике такой централизованный подход не масштабируется до десятков тысяч узлов и сервисов, которые Consul стремится поддерживать. Вместо этого, запуск Consul клиента на каждом узле позволяет использовать распределенный подход к проверке здоровья сервисов и узлов.



Consul клиенты

- Для обнаружения сбоев узлов Consul использует библиотеку под названием Serf, которая реализует алгоритм gossip (также известный как эпидемический алгоритм).
- Принцип работы этого алгоритма напоминает распространение слухов или вирусов: каждый клиент передает сообщения небольшому числу других клиентов, выбранных случайным образом (по умолчанию трем).
 - Эти клиенты, в свою очередь, выбирают еще по три случайных клиента для общения, и так далее. Благодаря такому подходу информация быстро распространяется по всему кластеру.





Consul клиенты

- В Serf клиенты постоянно проверяют друг друга. Если определяется, что клиент не работает, эта информация быстро распространяется по всему кластеру.
- Хотя теоретически распределенное обнаружение сбоев занимает больше времени, чем централизованный подход, на практике сбой обнаруживается и распространяется среди всех клиентов менее чем за две секунды, даже в кластерах из тысяч узлов.



Consul клиенты

- В среде Kubernetes Consul клиенты запускаются как DaemonSet - особый тип ресурса, который гарантирует, что на каждом узле кластера будет работать один под Consul клиента. Это обеспечивает постоянное наличие Consul клиента для управления sidecar прокси.
- При работе на виртуальных машинах администратор должен сам обеспечить наличие Consul клиента на каждом рабочем узле.



Consul - sidecar прокси

- Каждый экземпляр сервиса имеет свой выделенный sidecar прокси.
- Основная задача прокси - перехватывать запросы к сервису и от сервиса, а затем выполнять над этими запросами определенные действия согласно конфигурации. Например, прокси может шифровать запросы с помощью TLS или отклонять запросы от неавторизованных сервисов.



Consul - sidecar прокси

- Важно понимать, что сам Consul не является прокси. Вместо этого Consul использует прокси под названием Envoy - открытый проект, изначально созданный в компании Lyft. Сейчас Envoy является проектом Cloud Native Computing Foundation (CNCF) и используется многими service mesh решениями.
- Envoy написан на C++ для обеспечения высокой производительности и имеет очень небольшой объем потребляемой памяти. Это означает, что он может работать рядом со всеми вашими сервисами с минимальным влиянием на ресурсы и задержку. Envoy поддерживает сотни функций для балансировки нагрузки, наблюдаемости, проверки здоровья и многого другого. Consul клиенты настраивают Envoy через его gRPC API.



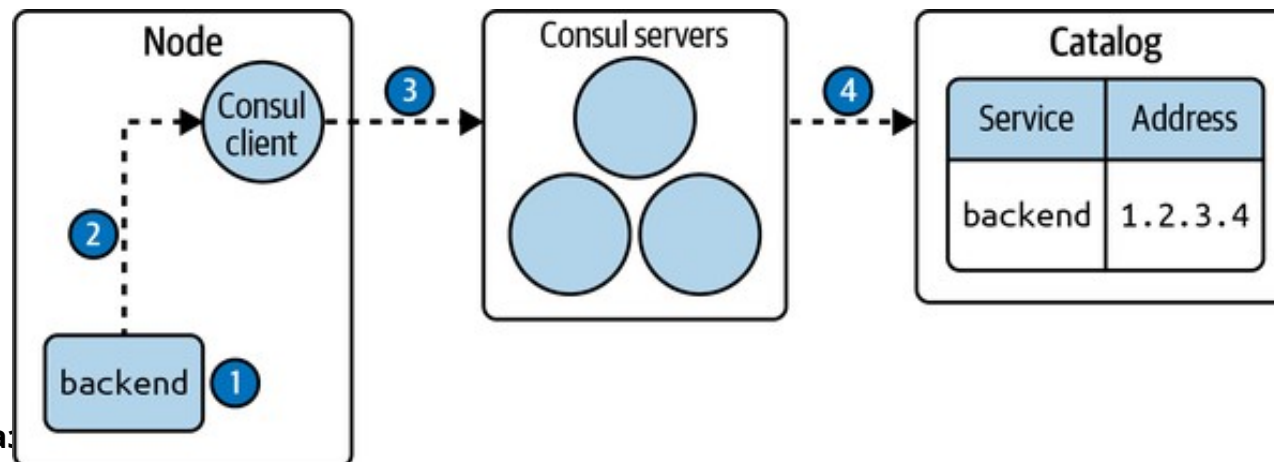
Consul - sidecar прокси

- В среде Kubernetes sidecar прокси работают как отдельные контейнеры внутри подов сервисов. Это гарантирует, что контейнеры сервисов могут взаимодействовать со своими sidecar прокси внутри локальной сети пода.
- При работе на виртуальных машинах sidecar прокси запускаются как отдельные процессы для каждого экземпляра сервиса на этой виртуальной машине.



Пример

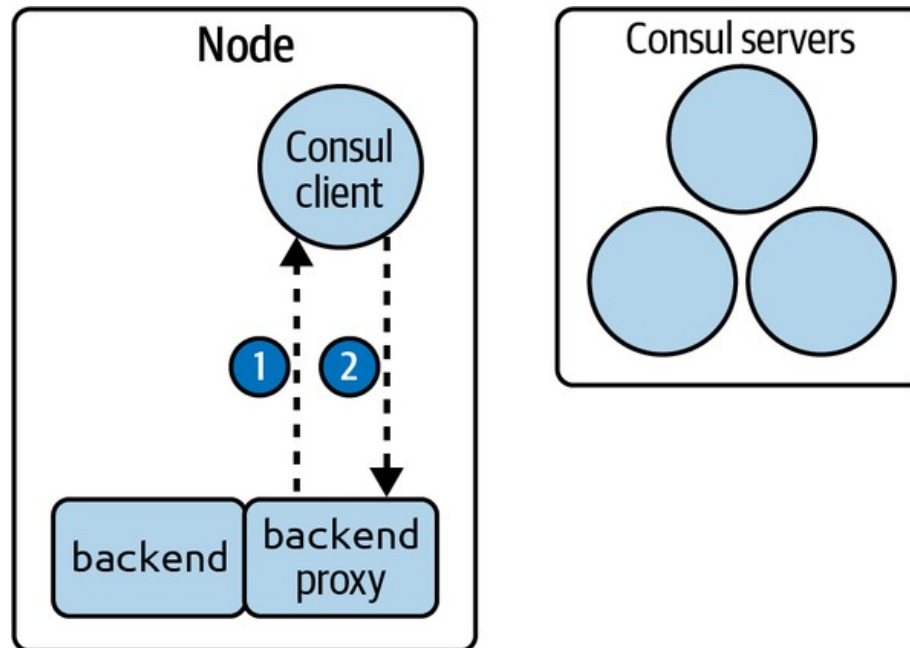
- Давайте схематический практический пример, как все эти компоненты работают вместе.
- Представим, что на узле запускается новый сервис под названием backend. Сначала этот сервис регистрируется со своим IP-адресом (например, 1.2.3.4) в Consul клиенте на этом узле. Затем Consul клиент отправляет запрос Consul серверам, чтобы сообщить им, что новый сервис backend работает на этом узле с указанным адресом. Consul серверы добавляют этот сервис в свой каталог.





Пример

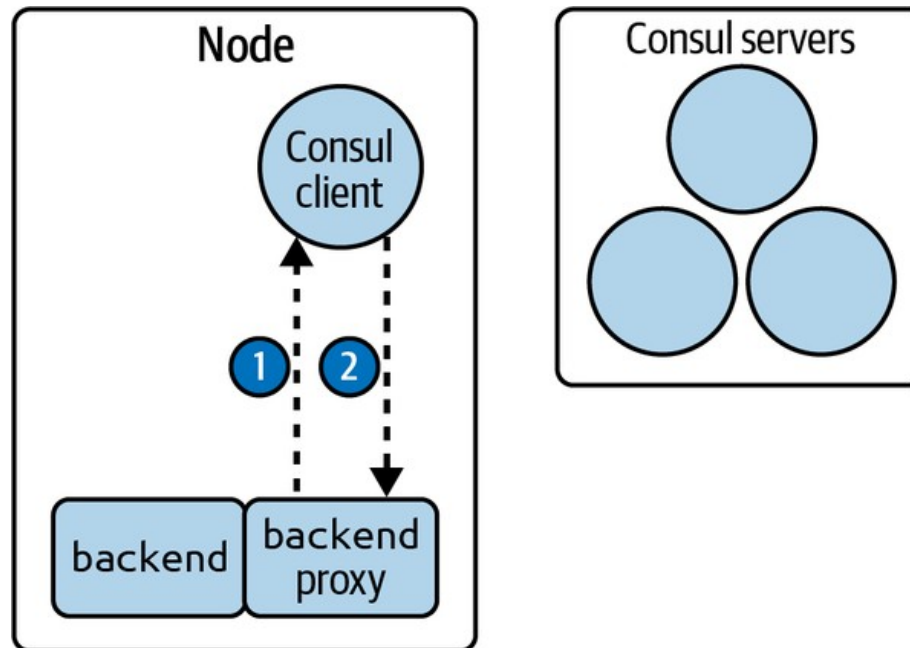
- Одновременно с этим запускается прокси для сервиса backend.
- Прокси открывает соединение с локальным Consul клиентом. Consul клиент настраивает прокси и поддерживает соединение открытым для будущих изменений конфигурации.





Пример

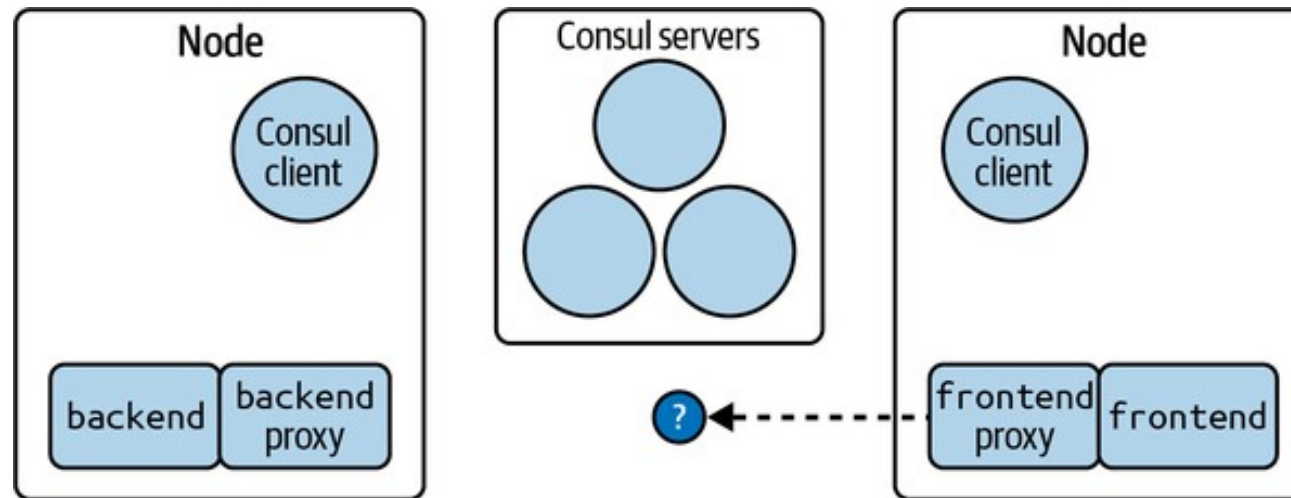
- Одновременно с этим запускается прокси для сервиса backend.
- Прокси открывает соединение с локальным Consul клиентом. Consul клиент настраивает прокси и поддерживает соединение открытым для будущих изменений конфигурации.





Пример

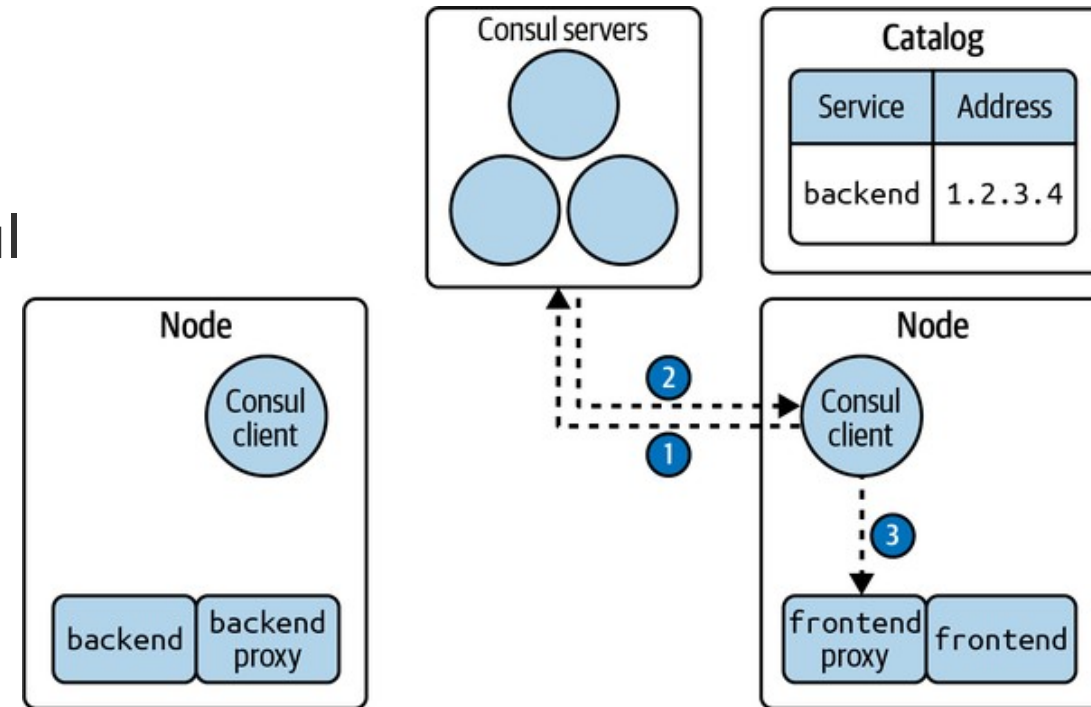
- Представим, что в нашем кластере есть еще один сервис под названием frontend, который работает на другом узле и должен взаимодействовать с нашим backend сервисом. Для того чтобы эта коммуникация состоялась, прокси frontend сервиса должен знать адрес backend сервиса.





Пример

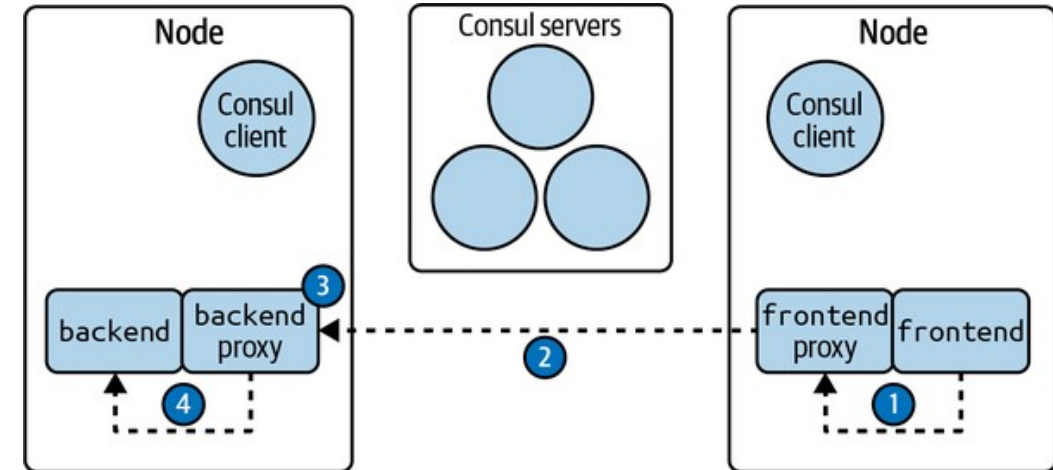
- Процесс обнаружения адреса происходит следующим образом: Consul клиент, находящийся на том же узле, что и frontend сервис, постоянно следит за каталогом Consul серверов на предмет появления новых экземпляров backend сервиса.
- Когда новый экземпляр backend сервиса регистрируется в каталоге, Consul серверы отправляют новый адрес Consul клиенту. После этого Consul клиент обновляет конфигурацию прокси frontend сервиса, добавляя туда новый адрес backend сервиса.





Пример

- Теперь, когда прокси frontend сервиса знает адрес backend сервиса, рассмотрим, как происходит сам процесс коммуникации.
- Когда frontend сервис делает запрос, его прокси перехватывает этот запрос. Прокси видит, что запрос предназначен для backend сервиса, и поскольку ему известен адрес backend сервиса (1.2.3.4), он перенаправляет запрос на этот адрес.
- Запрос достигает backend сервиса, где его перехватывает прокси backend сервиса. Предположим, что прокси backend настроен так, чтобы принимать запросы только от frontend сервиса. В этом случае прокси проверяет источник запроса и, убедившись, что запрос пришел от frontend сервиса, пропускает его дальше.





Consul

- Consul имеет ряд уникальных особенностей по сравнению с другими service mesh решениями, такими как Istio и Linkerd. Хотя большинство service mesh следуют одной и той же общей архитектуре с control plane, управляющей sidecar прокси, Consul выделяется тем, что его control plane может работать полностью независимо от Kubernetes.
- Это означает, что если вы управляете кластером виртуальных машин, вам не нужно дополнительно запускать Kubernetes.



Consul

- Важно отметить, что Consul - это не только service mesh. Он также предоставляет хранилище ключ-значение для конфигурации сервисов и решение для DNS service discovery.
- При использовании DNS не используются sidecar прокси, запросы направляются напрямую между сервисами. Это означает, что в таком случае нет автоматического шифрования, наблюдаемости или функций управления трафиком.



Будут вопросы? Не стесняйтесь!

